

PRIVATE RAG · INFERENCE · PRODUCTION LLMOPS

FinLLM Lab v0.2 최종 기술 보고서

단일 GPU 금융 Private RAG — 측정에서 운영 closed
loop까지

FINAL RELEASE · PASS

FinLLM Lab · 2026-08-09

Actual RTX A6000 integrated rehearsal · Native 24GB validation: NOT_EXECUTED

Contents

1. 1. Executive Summary
2. 2. 프로젝트를 시작한 이유
3. 3. 문제 정의
4. 4. 운영 제약
5. 5. FinLLM v0.1에서 만든 것
6. 6. v0.2에서 부족했던 것과 확장 이유
7. 7. 전체 Architecture
8. 8. 모델/양자화 선택 과정
9. 9. Evaluation 설계
10. 10. Serving / Deployment
11. 11. Observability
12. 12. SLO와 Alert
13. 13. Regression Gate
14. 14. Incident 실험
15. 15. Rollback
16. 16. Claude ↔ Codex Cross Review
17. 17. 주요 실패와 오진 — BEFORE → Investigation → AFTER
18. 18. 실제 해결 과정
19. 19. 최종 측정 결과의 해석
20. 20. 현재 시스템에서 아직 해결되지 않은 문제
21. 21. 이 프로젝트에서 배운 기술
22. 22. 다음 프로젝트: Quantization Autopsy
23. 23. 면접에서 설명한다면
24. Appendix A. 최종 검증 기록
25. Appendix B. 핵심 source map

1. Executive Summary

FinLLM Lab은 금융 내부분서를 다루는 단일 GPU Private RAG에서 모델 선택을 “더 큰 모델”이 아니라 **품질·지연·처리량·메모리·보안의 동시 제약**으로 다룬 프로젝트다. v0.1은 합성 corpus, deterministic retriever, ACL, 60문항 평가, result schema, load test와 GPU profile을 만들었다. v0.2는 이 측정 stack을 그대로 재사용해 Build → Deploy → Observe → Alert → Diagnose → Regression Test → Rollback → Evidence의 운영 loop를 추가했다.

최종 판정은 **PASS**다. release source 95dd24deba5669919e12b8535dbaf3128646ae5e 와 API image sha256:16986083...70feb 를 기준으로 actual NVIDIA RTX A6000 GPU 1에서 API, RAG, vLLM, Prometheus, Grafana, DCGM을 함께 실행했다. 153개 deterministic test, 27개 기존 result schema, 11개 CPU/GPU regression stage가 모두 통과했다. Prometheus target 4개가 up=1 이었고, 의도적 API 중단에서 alert firing과 immutable container recovery를 재현했다.

이 PASS는 범위가 있다. 운영 rehearsal은 A6000에서 실행했지만 과거 24GB-class 적합성 수치는 A6000의 memory-budget-emulation 이다. 실제 24GB GPU의 native-gpu-validation 은 **NOT_EXECUTED**다. 합성 16문서와 60문항은 실제 고객 corpus/traffic을 대체하지 않으며, prompt injection 5건 중 1건 성공이 남아 있다.

Release status: **PASS · single-A6000 v0.2 scope** Integrated A6000 rehearsal: **VERIFIED**
Native 24GB validation: **NOT_EXECUTED**

2. 프로젝트를 시작한 이유

금융 Private RAG는 일반 챗봇 데모와 실패 비용이 다르다. 내부분서를 외부 API로 보낼 수 없고, retrieval 단계에서 권한 없는 문서를 노출해도 안 된다. 한정된 GPU에서 여러 요청을 처리할 때 품질만 높거나 메모리에만 들어가는 구성도 운영 가능하다고 할 수 없다. benchmark 평균 하나보다 model, tokenizer, prompt, retriever, dataset revision이 더 중요하다.

프로젝트 질문은 다음과 같다.

이미 평가·추론 구성이 정해진 단일 GPU 금융 Private RAG를 재현 가능하게 배포하고, 운영 중 품질·성능 이상을 관측하고, 잘못된 변경을 차단하며, 장애 시 안전하게 복구할 수 있는가?

3. 문제 정의

측	질문	기준과 증거
품질	답이 맞고 근거가 있으며 필요한 경우 거부하는가	<code>scripts/rag_eval.py</code> , 60문항
보안	ACL 위반과 injection을 측정하는가	retrieval tests, injection cases
성능	concurrency 10에서 TTFT와 처리량은 어떤가	<code>scripts/load_test.py</code> , result JSON
메모리	executor 예산과 process peak를 구분하는가	<code>scripts/gpu_watch.py</code> , result schema
재현성	모든 입력 provenance가 남는가	configs, schemas, release manifest
운영	liveness/readiness, 관측, 장애 복구가 연결되는가	service, monitoring, INC-003
변경 안전성	실패·skip·stale 결과가 non-zero로 차단되는가	regression gate, negative tests

4. 운영 제약

4.1 On-premise와 single GPU

Kubernetes나 multi-node cluster는 범위가 아니다. 단일 RTX A6000 48GiB 부서 서비스를 Docker Compose로 재현하는 것이 목표다. 이 선택은 운영 복잡도를 줄이는 대신 GPU와 host가 single point of failure라는 한계를 남긴다.

4.2 GPU memory evidence 경계

`gpu_memory_utilization=0.50/0.46`은 A6000 executor 예산을 제한해 24GiB-class 조건을 탐색한 실험 장치다. A6000의 tok/s와 TTFT를 RTX 4090/5090 실측으로 부를 수 없고, 실제 24GB 카드에서의 native fit도 증명하지 않는다.

4.3 Security와 reproducibility

corpus는 합성 문서 16개다. ACL은 모델 프롬프트가 아니라 retrieval 전 필터에서 강제한다. 최종 model/tokenizer revision은 `31c69efc29464b6bb0aee1398b5a7b50a99340c3`, prompt는 `prompt-v0.1`, eval set은 `eval-v0.1`, retriever hash는 `11d1f8cfef42`다.

5. FinLLM v0.1에서 만든 것

- synthetic 금융 corpus 16개와 ACL metadata
- 60문항 evaluation set과 expected-fact 기반 scoring
- correctness, groundedness, citation, abstention 분해
- injection/unauthorized retrieval 진단
- deterministic sparse retriever와 config hash
- concurrent load test와 server/user latency 분리
- `memory-budget-emulation` 과 `native-gpu-validation` 을 구분하는 schema
- model/tokenizer/prompt/corpus/eval/retriever provenance
- GPU peak sampling과 반복 측정

6. v0.2에서 부족했던 것과 확장 이유

v0.1은 어떤 구성이 실험에서 나왔는지는 답했지만 다음 질문에는 답하지 못했다.

- process liveness와 traffic readiness가 분리되는가?
- model/index가 준비되지 않았을 때 silent fallback 없이 실패하는가?
- SIGTERM 후 새 요청을 막고 accepted request body까지 drain하는가?
- request/retrieval/generation 지표를 운영 중 수집하는가?
- threshold를 넘으면 실제 alert가 firing하는가?
- 품질이 나빠진 변경과 실행되지 않은 gate를 차단하는가?
- known-good immutable release로 복구한 뒤 readiness를 확인하는가?

7. 전체 Architecture

```
Client
  → FinLLM API /health · /ready · /metrics
    → deterministic ACL Retriever / frozen index
    → vLLM OpenAI API
      → NVIDIA RTX A6000 GPU 1

Prometheus ← API metrics · vLLM metrics · DCGM
  ↓
Grafana + alert → runbook → diagnose → rollback

CI / local gate → tests → schema → CPU/GPU eval → release manifest
```

API와 관측 stack은 external `finllm-net` 에서 연결된다. actual rehearsal에서 `finllm-api:8080`, `vllm:8000`, `dcgm-exporter:9400`, Prometheus 자체 target이 모두 `up=1` 이었다. 구조 증거는 `ops/evidence/final-rehearsal/prometheus-targets-ready.json` 에 있다.

8. 모델/양자화 선택 과정

아래는 2026-08-08c eager 결과 세 반복의 산술평균이다. 모두 **A6000 memory-budget-emulation**이며 concurrency 10, 30 requests, `max_model_len=8192`, `max_num_seqs=10` 조건이다.

모델/조건	Quality	server P95 TTFT	user P95 TTFT	output tok/s	peak VRAM	max conc. @8192	Evidence
Qwen3-8B BF16 · 0.50 eager	95.926	77.329ms	1,344.653ms	286.955	24.006GiB	7.31	A6000 emulation
Qwen3-14B AWQ · 0.50 eager	97.667	129.828ms	1,310.587ms	313.238	23.836GiB	11.05	A6000 emulation
Qwen3-14B AWQ · 0.46 eager	97.667	129.995ms	1,273.402ms	315.331	21.961GiB	9.53	A6000 emulation

0.46 AWQ는 같은 host/workload에서 높은 품질과 가장 낮은 관측 peak를 보였으므로 native 24GB 검증의 후보가 됐다. 9.53은 full 8192-token budget으로 계산한 capacity 지표이며 실제 요청 concurrency 10과 같은 뜻이 아니다.

9. Evaluation 설계

60문항은 answerable, abstention, unauthorized, injection으로 나뉜다. 단일 LLM judge 점수가 아니라 expected facts, citation, refusal 조건을 deterministic하게 계산한다. ACL violation은 전체 실행을 실패시킨다. v0.2 release gate 실제 재실행 결과는 quality 98.333, correctness 100.0, groundedness 95.0, citation 100.0, abstention 98.333, ACL violation 0, injection success 1이었다. 이는 한 번의 actual gate 결과이며 과거 baseline 97.667을 수정하지 않는다.

10. Serving / Deployment

항목	구현과 actual 검증
GPU container	CUDA base digest, Python 3.10.12, vLLM 0.9.2, torch 2.7.0+cu126, transformers 4.53.2 고정
Compose	API + vLLM을 GPU 1에서 기동; monitoring stack과 finllm-net 연결
/health	process liveness만 검사; actual HTTP 200
/ready	app/retriever/endpoint/model/admission 분리; actual ready/503 drain 전이
/metrics	요청/error/in-flight/request/retrieval/generation histogram과 build info
startup	config/directory/index/model/revision/hash 검증; failure tests 통과
shutdown	admission close 후 accepted response body까지 drain

실제 Compose의 API image ID는 sha256:16986083...70feb , vLLM image ID는 sha256:ec36cea2...99a3d 다. CUDA container는 기존 고정 base를 그대로 사용했고 CUDA나 host driver를 변경하지 않았다.

11. Observability

Prometheus는 application, vLLM, DCGM metric을 수집한다. 안정 application contract는 다음 여섯 이름이다.

```
finllm_requests_total
finllm_request_errors_total
finllm_requests_in_flight
finllm_request_duration_seconds
finllm_retrieval_duration_seconds
finllm_generation_duration_seconds
```

추가로 `finllm_ready`, `finllm_shutdown_in_progress`, `finllm_build_info`가 readiness와 provenance를 제공한다. Grafana 11.3.1에서 `FinLLM Service – Profile A` dashboard와 Prometheus datasource provisioning을 API로 확인했다. dashboard 존재와 실제 metric 수집을 분리해 두 증거를 모두 보존했다.

12. SLO와 Alert

hard policy는 quality ≥ 90 , P95 TTFT $\leq 2,000\text{ms}$, error rate $\leq 1\%$, OOM 0, concurrency 10이다. 10개 alert rule은 promtool 로 검증했다. error ratio는 `finllm_request_errors_total / finllm_requests_total` rate로 실제 contract와 일치한다. vLLM TTFT histogram은 2.0초 bucket이 없어 `[1.0, 2.5]` 보간이며 release 판정이 아닌 조기 신호로만 사용한다.

for: 와 rate window는 아직 실제 장기 traffic 분포로 정당화하지 못해 rule annotation에 `PENDING_THRESHOLD_VALIDATION` 을 유지했다. 반면 service-down alert는 INC-003에서 실제 firing을 확인했다.

13. Regression Gate

CPU stage는 unit test, result schema, eval integrity, retrieval ACL, retriever hash, prompt revision, alert threshold를 검사한다. GPU stage는 live 60문항 평가, runtime ACL, quality, injection regression을 검사한다.

최종 actual 결과는 `pass=11`, `fail=0`, `skipped=0` 이다. quality 98.333은 policy 90과 baseline floor 97.667을 통과했다. stale output을 쓰지 않도록 매 실행 unique 경로를 사용하고 subprocess non-zero를 즉시 실패로 처리한다. 모든 stage를 skip하거나 `--stage all`에 skip이 있으면 exit 1이다. 이 negative path는 unit test에 포함됐다.

첫 actual 전체 gate는 평가가 성공했지만 상대 evidence path 처리 예외로 FAIL했다. 결과를 통과로 덮지 않고 경로를 정규화한 뒤 전체 gate를 다시 실행해 PASS를 얻었다.

14. Incident 실험

INC-003은 실제 고객 장애가 아니라 의도적 **loopback** 실험이다. actual A6000 stack에서 20개의 RAG 요청이 진행 중일 때 API에 SIGTERM을 보냈다.

단계	실제 관측
drain	새 요청 수락 중단, /ready=503
accepted requests	20/20 HTTP 200 body 완주, 실패 0
API 중단	Prometheus gateway target down
alert	FinLLMGatewayDown firing, SIGTERM 후 43.765초
recovery	active alert 없음, /ready=200

상세 timeline과 원인/영향 경계는 ops/incidents/INC-003-api-outage-container-rollback.md 에 있다.

15. Rollback

known-good manifest는 full Git SHA, API/vLLM image ID, model/tokenizer revision, prompt/eval/retriever provenance, all-stage gate report를 기록한다. rollback 순서는 다음과 같다.

```
manifest/schema/gate 검증
→ local image digest와 deploy provenance 대조
→ Compose restart
→ /v1/models + /ready + finllm_build_info 검증
→ current release와 audit log commit
```

첫 복구 시도는 manifest Git SHA 오기 때문에 restart script가 exit 1로 차단했고 `state_mutated=false`로 기록됐다. 실제 SHA로 수정한 성공 실행은 6.332초였으며 검증이 끝난 뒤에만 current release가 변경됐다. 독립 verify도 VERIFY: OK였다.

16. Claude ↔ Codex Cross Review

Codex는 Serving/Deployment, Claude는 Observability/Reliability ownership을 맡고 interface contract로 연결했다. 교차검토는 generation 실패 뒤 readiness 200, response body 전달 전 in-flight 감소, 작은 listen backlog, stale/skip gate fail-open, failed restart state mutation을 찾았다. finding은 LLM 의 견으로 합격시키지 않고 재현 스크립트와 deterministic test로 다시 판정했다.

```
LLM = 구현자와 검토자  
Contract = 협업 경계  
Reproduction = finding 근거  
Test · schema · actual run = 최종 judge
```

17. 주요 실패와 오진 — BEFORE → Investigation → AFTER

사례 A — AWQ throughput 오진

BEFORE: graph-enabled AWQ의 약 57.2 tok/s를 보고 dequantization overhead를 의심했다. **Problem:** weight format과 CUDA execution path를 분리하지 못한 설명이었다. **Investigation:** model/revision/eval을 고정하고 `--enforce-eager` 하나만 변경해 세 번 재측정했다. **AFTER:** 14B AWQ 평균은 313.238 tok/s가 됐다. 정확한 kernel root cause는 NOT_MEASURED이므로 “graph-enabled path와 함께 나타난 현상”까지만 결론냈다.

사례 B — component PASS와 release safety 혼동

BEFORE: A/B 각자의 unit test PASS가 closed loop 완성을 뜻하는 것처럼 보였다. **Problem:** 실제 generation failure, HTTP body drain, stale evaluation, skip-only gate, failed rollback 경로는 검증되지 않았다. **Investigation:** 각 결함을 독립 재현하고 fail-open 결과를 기록했다. **AFTER:** readiness latch, response lease, unique eval output, executed-stage minimum, verification-before-state-commit으로 수정했고 actual A6000 incident에서 다시 검증했다.

18. 실제 해결 과정

1. A service와 B monitoring을 canonical repository/contract 하나로 통합했다.
2. API metric 이름과 PromQL/Grafana query를 실제 exposition에 맞췄다.
3. generation failure latch와 body-delivery request lease를 구현했다.
4. regression/rollback negative path를 fail closed로 바꿨다.
5. pinned image를 build하고 GPU 1에서 full Compose를 기동했다.
6. 60문항 all-stage gate, graceful drain, alert, rollback을 실제 실행했다.
7. 모든 evidence를 release manifest와 incident report에 연결했다.

19. 최종 측정 결과의 해석

결과	값	의미 경계
deterministic suite	153 tests PASS	GPU opt-in 3개는 별도 actual PASS
existing result schema	27/27 PASS	v0.1 semantics 유지
all-stage gate	11/11 PASS	60문항, skipped 0
release-gate quality	98.333	단일 actual run; baseline 기록을 덮지 않음
graceful drain	20/20 complete	loopback actual API requests
gateway-down detection	43.765s	for:30s + scrape 정렬 포함
successful rollback command	6.332s	actual customer downtime과 다름
startup GPU 1 FB used	22,362MiB	snapshot; integrated load peak 아님

과거 0.46 AWQ 평균 315.331 tok/s, server P95 129.995ms, peak 21.961GiB는 v0.1 load/profile 결과다. v0.2 API를 거친 latency/throughput benchmark로 바뀌 부르지 않는다.

20. 현재 시스템에서 아직 해결되지 않은 문제

1. prompt injection success 1/5를 줄이는 방어 실험이 필요하다.
2. for: 와 rate window는 실제 traffic distribution으로 재측정해야 한다.
3. actual 24GB GPU native validation은 실행하지 않았다.
4. remote self-hosted A6000 CI job은 workflow만 존재하고 GitHub에서 실행하지 않았다.
5. rollback verifier가 runtime vLLM container digest를 자동 대조하는 기능은 후속 개선이다.
6. single-host Compose이므로 high availability와 무중단 model swap은 범위 밖이다.
7. 합성 16문서/60문항에서 실제 금융 corpus로의 외적 타당성은 별도 승인이 필요하다.

21. 이 프로젝트에서 배운 기술

- GPU executor budget, process peak, native card fit은 서로 다른 증거다.
- TTFT는 server scheduling, queue wait, user-observed latency를 분리해야 한다.
- quantization 문제는 weight format과 execution mode를 통제해 실험해야 한다.
- readiness는 liveness가 아니며 dependency와 admission state를 포함한다.
- Prometheus는 metric name뿐 아니라 type, labels, histogram buckets가 contract다.
- regression gate의 핵심은 정상 PASS보다 stale/skip/crash 시 fail closed다.
- rollback은 immutable target, restart, readiness, state commit 순서다.
- provenance가 없으면 정확해 보이는 숫자도 재사용 가능한 evidence가 아니다.

22. 다음 프로젝트: Quantization Autopsy

- eager/graph path에서 어떤 kernel과 shape가 병목인가?
- quantized weight의 memory saving이 KV cache/concurrency로 어떻게 이동하는가?
- prompt/output length와 concurrency가 TTFT/decode throughput을 어떻게 갈라놓는가?
- Ampere A6000 현상이 다른 architecture의 native GPU에서도 재현되는가?
- Nsight Systems/Compute와 vLLM metric을 어떤 provenance schema로 연결할 것인가?

다음 단계에서도 “AWQ가 빠르다/느리다”가 아니라 조건별 causal hypothesis를 만들고 실행하지 않은 native 결과를 추정값과 분리해야 한다.

23. 면접에서 설명한다면

30초

“단일 RTX A6000 금융 RAG에서 품질, TTFT, 처리량, VRAM, ACL을 동일 provenance로 비교했습니다. AWQ 저성능의 초기 해석을 eager 통제 실험으로 수정하고, pinned Compose, Prometheus, 회귀 gate, incident와 immutable rollback까지 실제 실행으로 연결했습니다. AI agent는 구현과 검토에 썼지만 최종 판단은 test, schema, command output과 actual GPU evidence로 내렸습니다.”

가장 먼저 보여줄 세 가지

1. 2026-08-08c result와 ADR-0004: 오진을 추가 실험으로 수정한 과정.
2. INC-003: 20/20 drain → alert firing → fail-closed rollback → readiness recovery.
3. gate-all.json: 기존 27개 result semantics를 유지한 11-stage actual gate.

Appendix A. 최종 검증 기록

검증	결과
deterministic tests	VERIFIED — 153 PASS, 3 opt-in GPU tests 별도 PASS
result schema	VERIFIED — 27/27
Compose build/start	VERIFIED — actual A6000 GPU 1
health/ready/metrics/RAG	VERIFIED — actual HTTP
Prometheus/Grafana/DCGM	VERIFIED — 4 targets up, dashboard provisioned
all-stage regression	VERIFIED — 11/11, 60 cases, skipped 0
graceful shutdown	VERIFIED — 20/20 body complete, drain 503
service-down alert	VERIFIED — firing after 43.765s
immutable rollback	VERIFIED — 6.332s command + readiness/build-info verify
native 24GB GPU	NOT_EXECUTED
remote GPU CI	NOT_EXECUTED

Appendix B. 핵심 source map

- Final judge: docs/final-review/final-release-review.json
- Release gate: ops/evidence/final-rehearsal/gate-all.json
- Service/GPU evidence: ops/evidence/final-rehearsal/
- Incident: ops/incidents/INC-003-api-outage-container-rollback.md
- Release manifest: ops/release/history/2026-08-09-v02-container-good.json
- Result contract: schemas/run-result.schema.json
- Policy: configs/profiles.json
- Cross-review contract: docs/cross-review/interface-contract-v0.2.md
- Deployment: deploy/compose.yaml, service/
- Monitoring: monitoring/